Министерство науки и высшего образования Российской Федерации
Московский государственный университет имени М. В. Ломоносова
Механико-математический факультет
Кафедра вычислительной математики

КУРСОВАЯ РАБОТА

Анализ алгоритма восстановления облака точек в трехмерном пространстве и построения по нему минимальных многогранников по горизонтальным зашумленным точечным сечениям тела

Выполнил:
студент 5 курса
Дьяченко Алексей

Научный руководитель:
Валединский Владимир Дмитриевич

Москва
2026

Содержание

1 Введение **3**

2 Постановка задачи **4**
2.1 Основная идея метода 4
2.2 Определение максимума 4

3 Генерация горизонтальных контуров и моделирование шума **9**
3.1 Исходная модель 9
3.2 Формирование проекций 9
3.3 Построение горизонтального контура 9
3.4 Моделирование шума 10
3.5 Результат генерации 10

4 Подавление шума методом наименьших квадратов **11**
4.1 Исходные данные 11
4.2 Аппроксимация рядом Фурье 11
4.3 Метод наименьших квадратов 11
4.4 Метод «ближайших соседей» 12
4.5 Результат и сравнение с точной формой 12

5 Сравнение восстановленной и точной формы **13**

6 Трехмерные тела: генерация сечений, восстановление и подавление шума **20**
6.1 Исходные тела 20
6.2 Генерация горизонтальных точечных сечений 21
6.3 Восстановление эталонных тел 21
6.4 Восстановление зашумленных моделей 22
6.5 Подавление шума по слоям 23
6.6 Сравнение с эталоном 24
6.7 Минимальный многогранник: автоматический режим 25

7 Реальные данные **29**
7.1 Данные и подготовка 29
7.2 Два режима восстановления 29
7.3 Зашумленные данные 30
7.4 Сглаживание 30
7.5 Вывод по реальным данным 32

8 Программа «Огранка» **33**

9 Вывод **35**

10 Заключение **36**

Список литературы **37**

1 Введение

В компьютерном моделировании и геометрии часто встречается задача построения облака точек и восстановления 3D-моделей реальных объектов и ландшафтов. Одна из современных технологий для решения подобных задач носит название фотограмметрия. В данной статье мы продолжим вырабатывать структуры методологии для решения данной задачи, рассмотрев ее на геометрических примитивах.

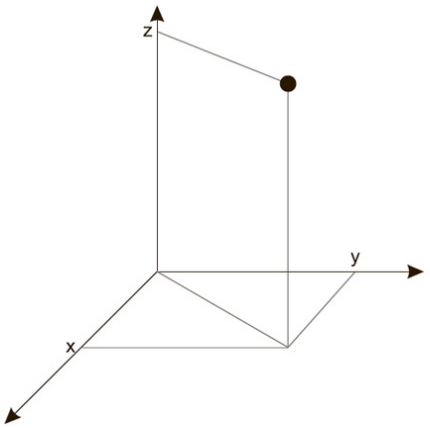


Рис. 1: Положение точки в трехмерном пространстве

2 Постановка задачи

Рассматривается задача восстановления вершин выпуклого многоугольника на фиксированной высоте z по его опорной функции вращения $r(\alpha)$, заданной дискретно.

Опорная функция выпуклого многоугольника — это расстояние от начала координат до опорной прямой с нормалью $e(\alpha) = (\cos\alpha, \sin\alpha)$, взятое в направлении нормали. Опорная прямая касается многоугольника в наиболее удаленной вершине, поэтому значение функции есть максимум проекций всех вершин:

$$r(\alpha) = \max_{j=1,\ldots,n} \left(x_j \cos\alpha + y_j \sin\alpha\right),$$

где (x_j, y_j) — координаты вершин.

При фиксированном направлении величина $x\cos\alpha + y\sin\alpha$ есть проекция точки (x,y) на это направление; именно такие проекции измеряются в задаче. Если α_i — направление максимума, то вершина лежит на опорной прямой и восстанавливается по значению функции:

$$x_i = r(\alpha_i)\cos\alpha_i, \qquad y_i = r(\alpha_i)\sin\alpha_i.$$

В точке максимума производная $r'(\alpha) = -x\sin\alpha + y\cos\alpha$ обращается в нуль, откуда $x_i/\cos\alpha_i = y_i/\sin\alpha_i = r(\alpha_i)$ — полезное соотношение для проверки. Требуется восстановить вершины по дискретно заданным значениям $r(\alpha_k)$.

2.1 Основная идея метода

Рассматривается функция $r(\alpha)$, заданная дискретно. Вершины многоугольника определяются как локальные максимумы этой функции.

2.2 Определение максимума

Для каждой точки i вычисляются разности:

$$d_1 = r_i - r_{i-1}, \quad d_2 = r_{i+1} - r_i.$$

Точка считается локальным максимумом, если:

$$d_1 \geq 0 \quad \text{и} \quad d_2 < 0.$$

Для визуализации сгенерированных входных данных была написана программа *visualize.py*, для выходных – *reconstruct.py*. Ниже приведены результаты тестирования на точных данных.

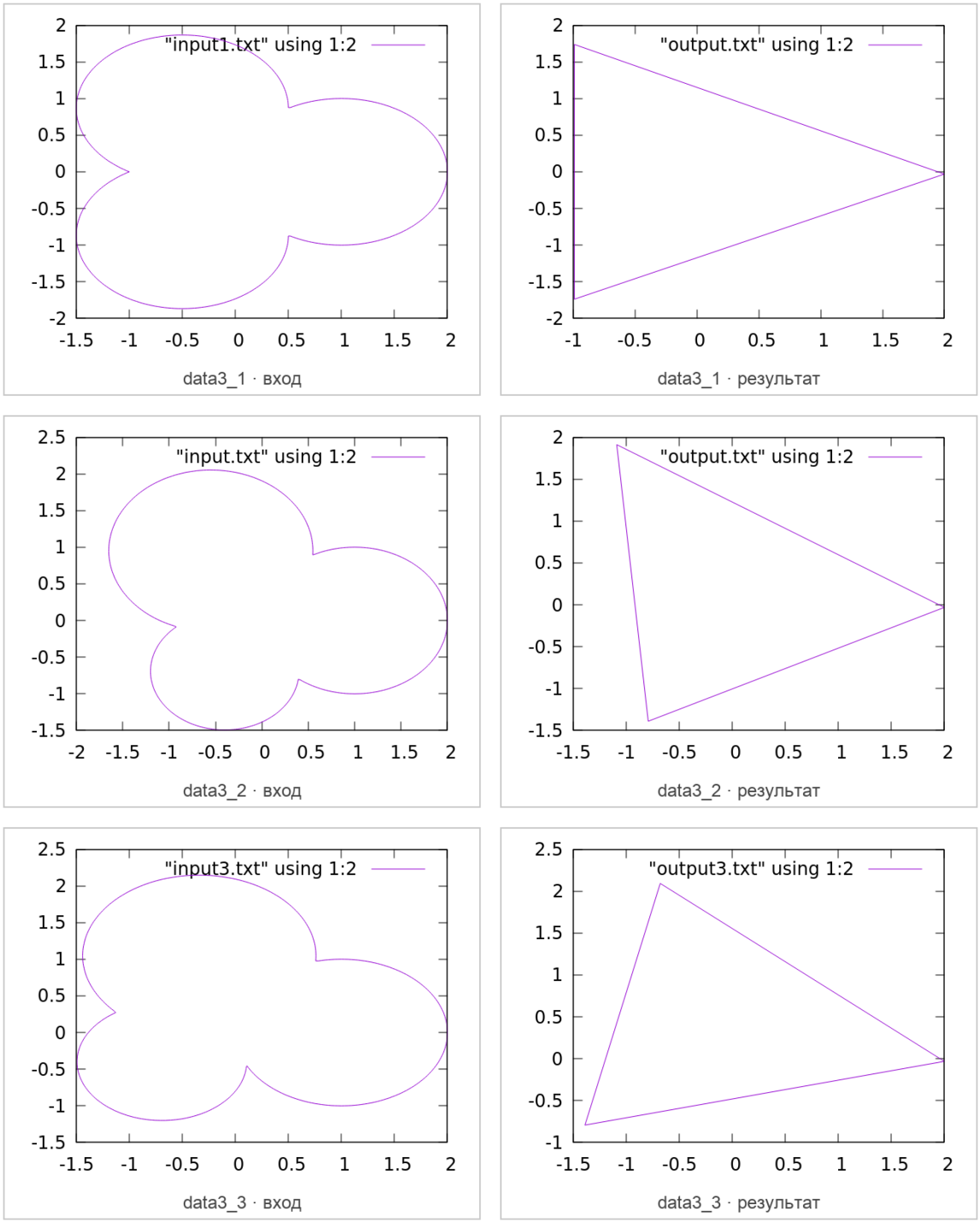


Рис. 2: Точные данные data3_1, data3_2, data3_3: слева входной контур, справа восстановленный многоугольник

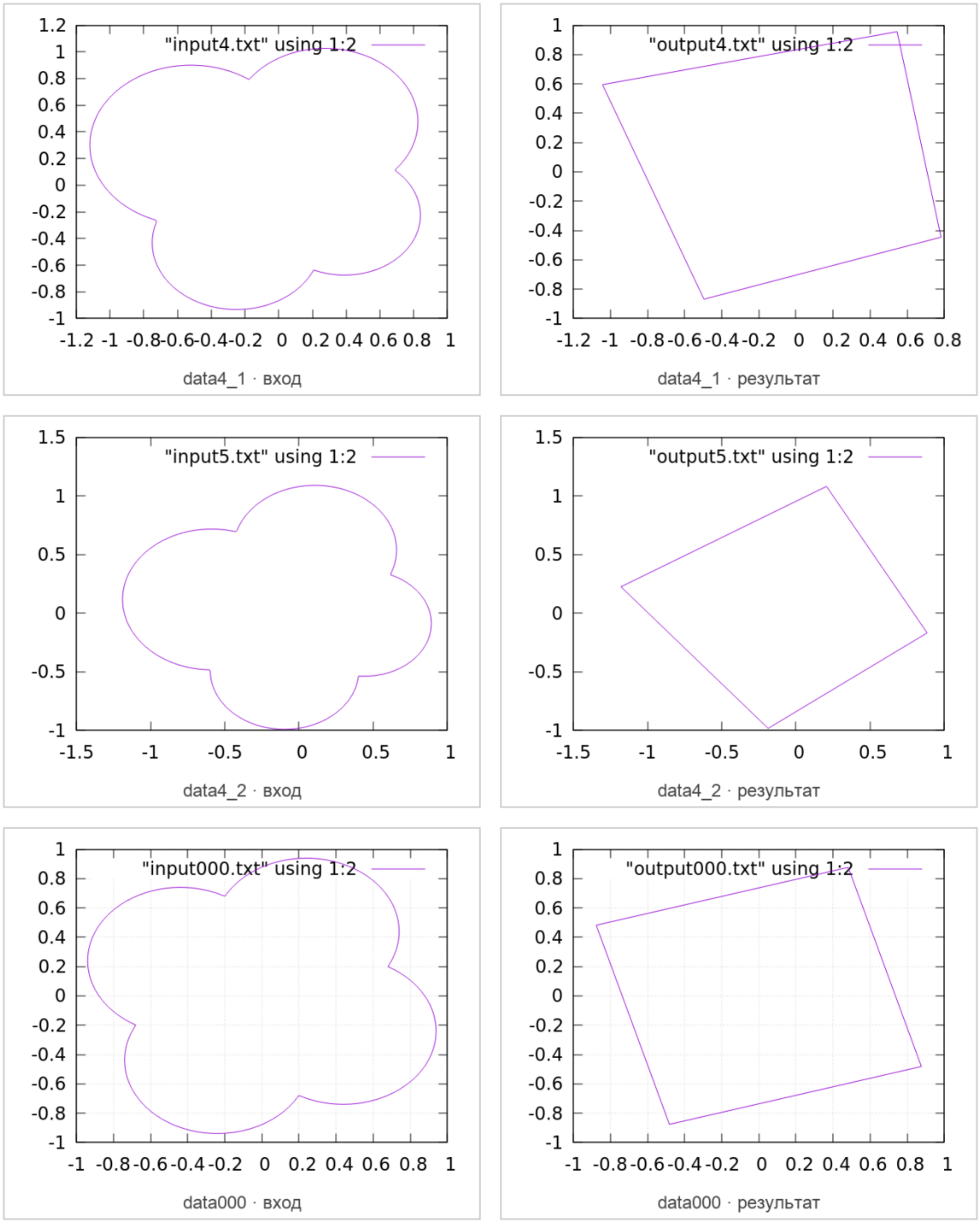


Рис. 3: Точные данные data4_1, data4_2, data000: слева входной контур, справа восстановленный многоугольник

Также протестировано на других данных ниже.

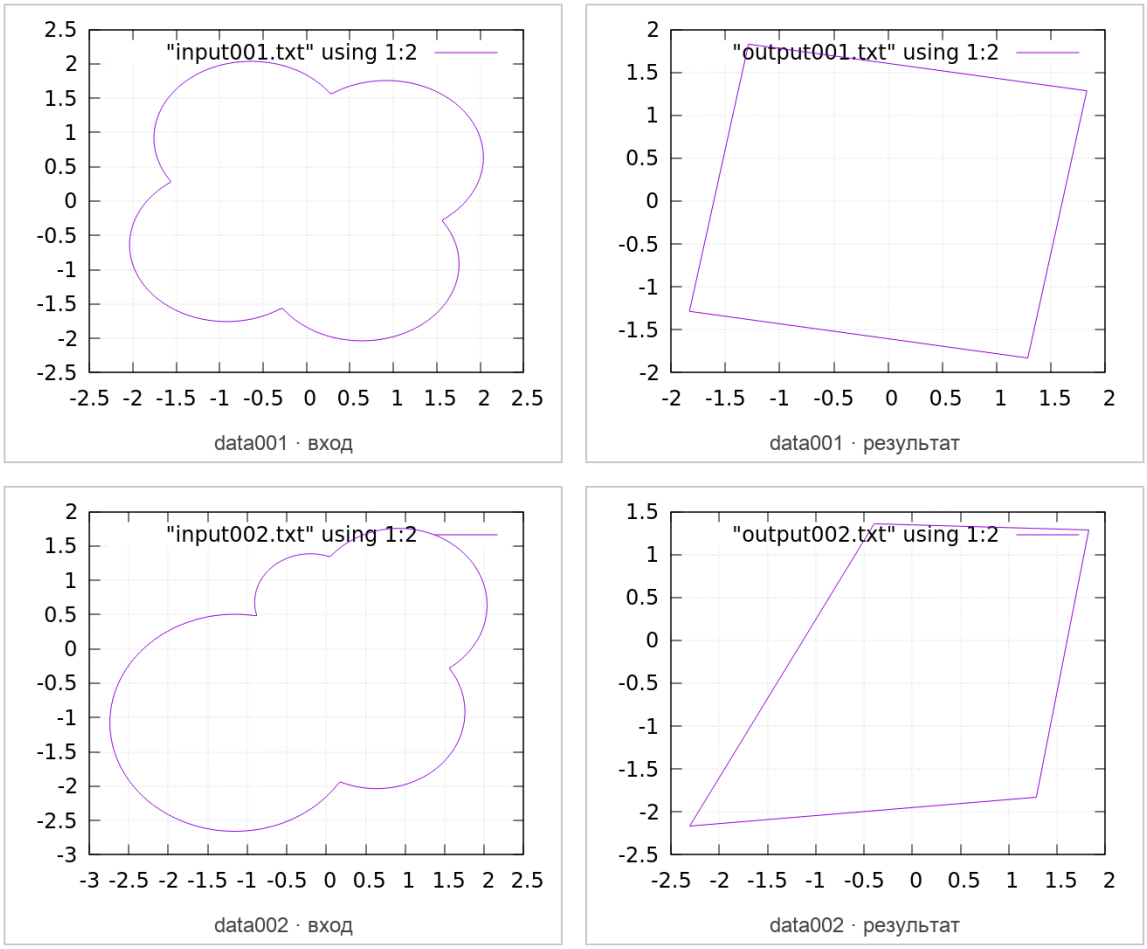


Рис. 4: Точные данные data001, data002: слева входной контур, справа восстановленный многоугольник

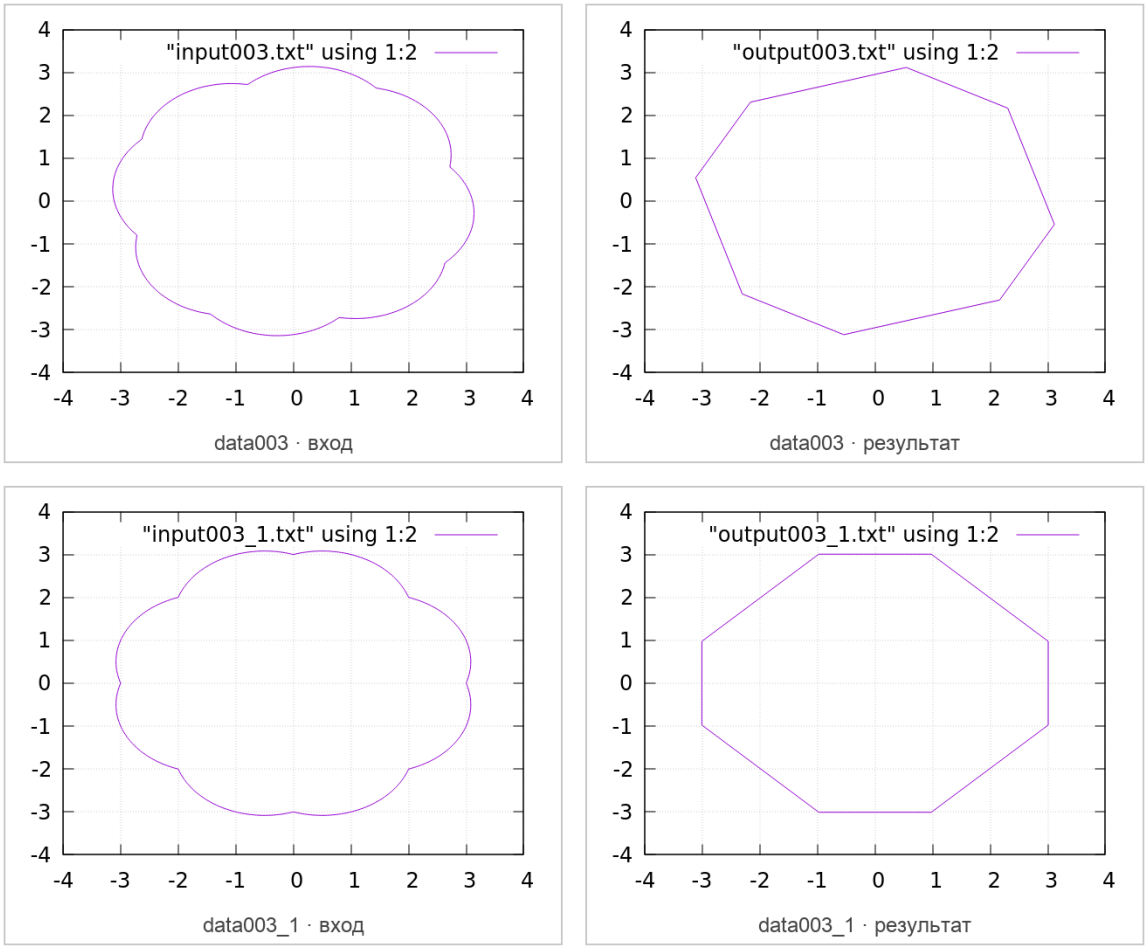


Рис. 5: Точные данные data003, data003_1: слева входной контур, справа восстановленный многоугольник

3 Генерация горизонтальных контуров и моделирование шума

Для тестирования алгоритмов восстановления формы объекта использовалась процедура генерации горизонтальных контуров выпуклого многоугольника.

3.1 Исходная модель

Исходный контур задаётся набором вершин:

$$P_k = (x_k, y_k).$$

В программе для генерации data003_1.txt использовался восьмиугольник:

$$(3,1),\ (1,3),\ (-1,3),\ (-3,1),\ (-3,-1),\ (-1,-3),\ (1,-3),\ (3,-1).$$

3.2 Формирование проекций

Для каждого направления проекции вычисляется расстояние от начала координат до опорной прямой.

Угол проекции определяется как:

$$\alpha_i = ih,$$

где

$$h = \frac{2\pi}{n},$$

а n — число проекций.

Для каждой вершины вычисляется:

$$r = \sqrt{x^2 + y^2},$$

$$\varphi = \arctan 2(y, x).$$

Затем для каждого угла проекции определяется значение:

$$p_i = r\cos(\alpha_i + \varphi).$$

Данная величина представляет собой проекцию вершины на направление наблюдения.

3.3 Построение горизонтального контура

Контур формируется как максимум среди проекций всех вершин:

$$R(\alpha_i) = \max_k p_i^{(k)}.$$

Таким образом, для каждого угла выбирается наиболее удалённая точка силуэта.

Полученная функция:

$$R(\alpha)$$

является дискретной опорной функцией выпуклого многоугольника.

3.4 Моделирование шума

Для имитации ошибок измерения в проекции добавляется случайный шум:

$$\widetilde{p_i} = p_i + \varepsilon,$$

где

$$\varepsilon = r_v \cdot \xi,$$
$$\xi \in [-1, 1].$$

Параметр:

$$r_v$$

задаёт амплитуду шума.

В работе использовались значения (в названиях файлов далее r8 и r80 соответственно):

$$r_v = 0.008,\ 0.08.$$

Шум моделирует:

- ошибки измерений,
- дискретизацию,
- неточности восстановления контуров.

3.5 Результат генерации

В результате формируется набор данных:

$$(\alpha_i,\ R(\alpha_i)),$$

который записывается в текстовый файл и далее используется для восстановления геометрии объекта.

4 Подавление шума методом наименьших квадратов

Для уменьшения влияния шума использовалась аппроксимация опорной функции при помощи ряда Фурье и метода наименьших квадратов, а также вспомогательного метода "ближайших соседей" (взвешенного скользящего среднего).

4.1 Исходные данные

После генерации контуров формируется набор значений:

$$(\alpha_i, r_i),$$

где:

- α_i — угол проекции,
- r_i — расстояние до опорной прямой.

Из-за добавления случайного шума значения содержат ошибки:

$$r_i = r_i^{\text{true}} + \varepsilon_i.$$

4.2 Аппроксимация рядом Фурье

Поскольку функция:

$$r(\alpha)$$

является периодической, она представляется рядом Фурье:

$$r(\alpha) \approx a_0 + \sum_{k=1}^{m} \left(a_k \cos(k\alpha) + b_k \sin(k\alpha)\right),$$

где:

- m — число гармоник,
- a_k, b_k — коэффициенты разложения.

4.3 Метод наименьших квадратов

Для вычисления коэффициентов строится система:

$$Ac \approx r,$$

где:

$$A = \begin{bmatrix} 1 & \cos(\alpha_1) & \sin(\alpha_1) & \dots \\ 1 & \cos(\alpha_2) & \sin(\alpha_2) & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix},$$

$$c = \begin{bmatrix} a_0 & a_1 & b_1 & \dots \end{bmatrix}^T.$$

Коэффициенты определяются из условия минимизации квадратичной ошибки:

$$\|Ac - r\|^2 \to \min.$$

Решение вычисляется методом наименьших квадратов:

$$c = (A^T A)^{-1} A^T r.$$

4.4 Метод «ближайших соседей»

Другой вспомогательный метод, который может дать лучшие приближения (ниже число итераций k – количество последовательных усреднений):

$$\widetilde{r}(\alpha_i) = \frac{2r(\alpha_i) + r(\alpha_{i-1}) + r(\alpha_{i+1})}{4}.$$

Для данного метода была написана программа *denoise_uniform.py* с автоматической регулировкой числа k итераций усреднения при сравнении с эталоном (для сглаживания был установлен нижний порог $k = 5$):

$$\mathrm{RMSE}(k) = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(r_i - \widetilde{r}_i^{(k)}\right)^2}$$

$$k_{\mathrm{opt}} = \arg\min_k \mathrm{RMSE}(k)$$

4.5 Результат и сравнение с точной формой

После вычисления коэффициентов получается сглаженная функция:

$$\widetilde{r}(\alpha),$$

которая приближает исходную форму объекта и подавляет высокочастотный шум. Для этого была написана программа *denoise.py*.

Полученные данные далее используются для восстановления геометрии многоугольника.

5 Сравнение восстановленной и точной формы

Для оценки качества восстановления выполнялось сравнение координат восстановленных точек с координатами точной модели.

Для каждой пары точек вычислялось евклидово расстояние:

$$d_i = \sqrt{(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2},$$

где:

- (x_i, y_i) — точка точной модели,
- $(\hat{x}_i, \hat{y}_i)$ — восстановленная точка.

Качество восстановления оценивалось двумя величинами.

Среднеквадратичная ошибка:

$$\mathrm{MSE} = \frac{1}{N} \sum_{i=1}^{N} d_i^2.$$

Максимальное отклонение:

$$d_{\max} = \max_i d_i.$$

Полученные значения позволяют оценить точность восстановления формы после подавления шума. Для этого была написана программа *compare.py*.

Ниже результаты тестирования на восьмиугольнике, параметр шума в названии файла. Также результаты тестирования в 3D, где сечения взяты на разных высотах.

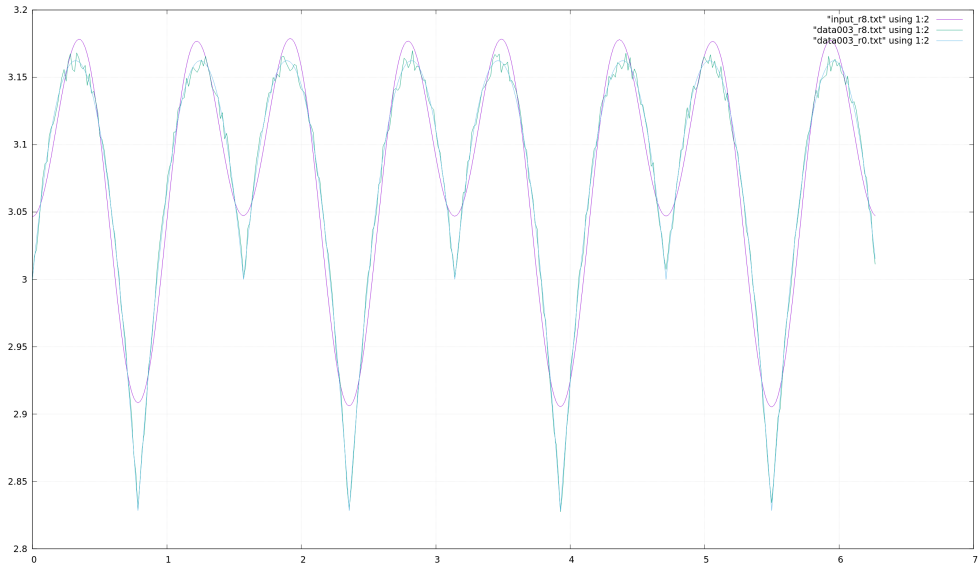


Рис. 6: Сравнение зашумленного и сглаженного варианта $r(\alpha)$ с эталоном

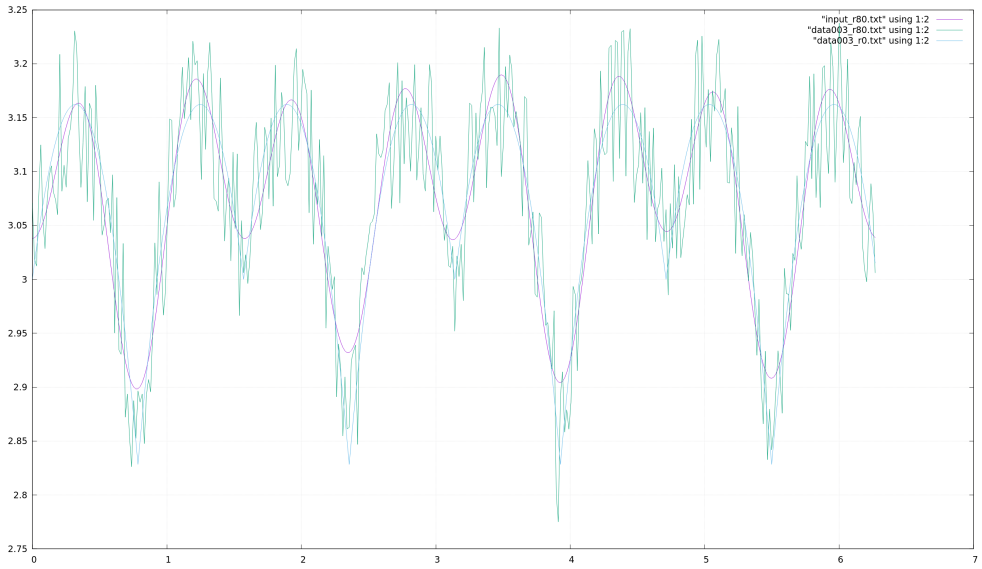


Рис. 7: Сравнение сильно зашумленного и сглаженного варианта $r(\alpha)$ с эталоном

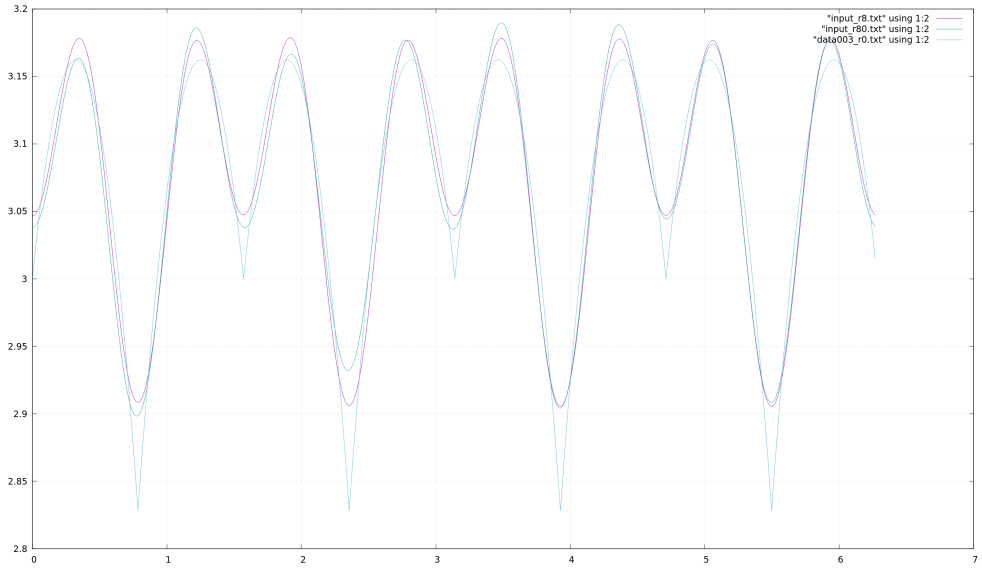


Рис. 8: Сравнение сглаженных результатов с эталоном

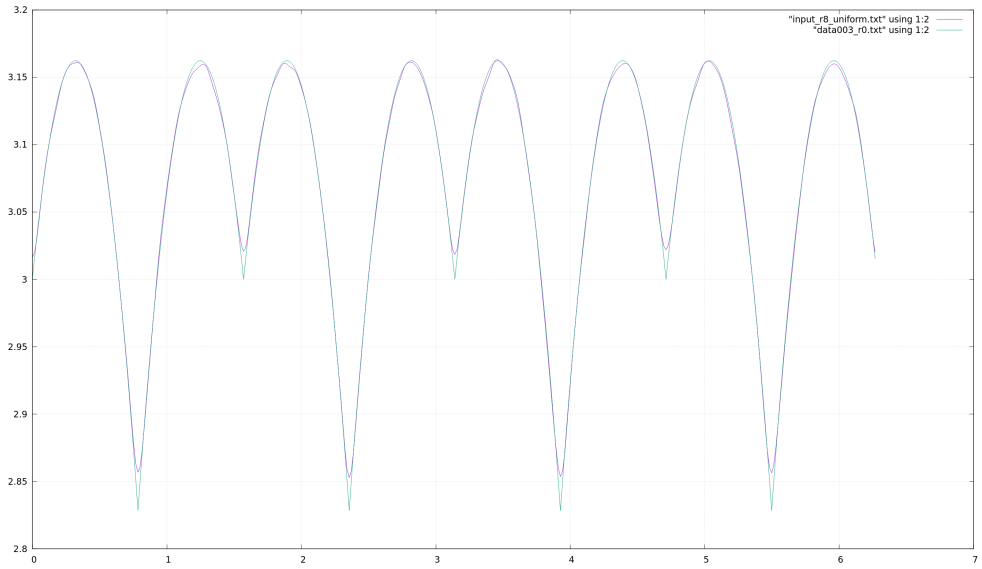


Рис. 9: Метод "ближайших соседей" для r8 ($k = 5$)

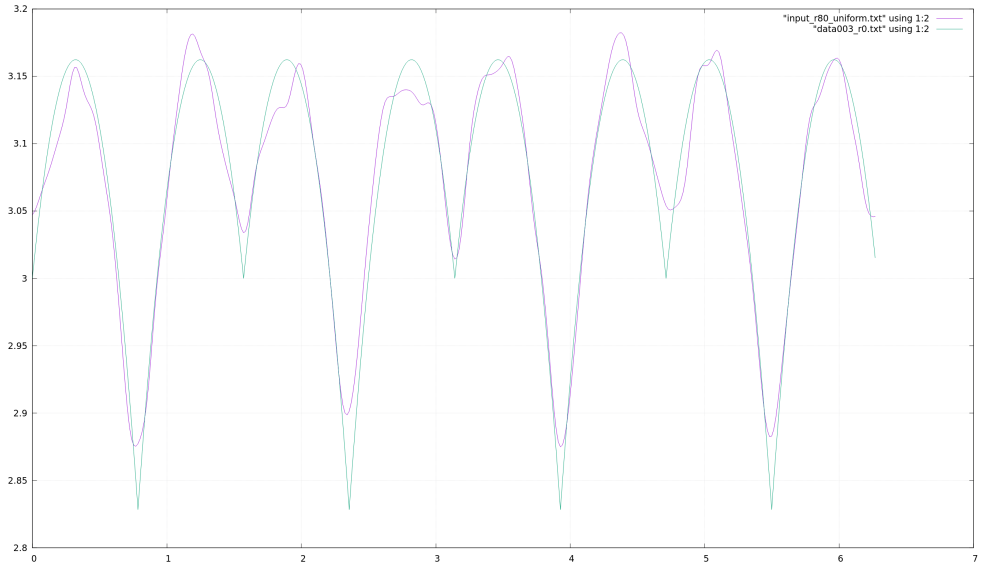


Рис. 10: Метод "ближайших соседей" для r80 ($k = 15$)

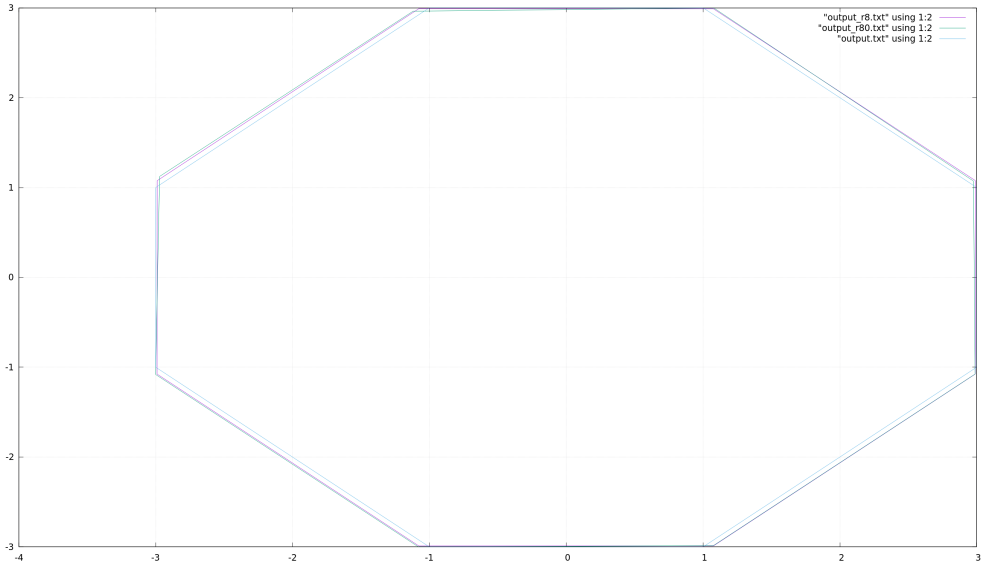


Рис. 11: Сравнение восстановленных многоугольников с эталоном

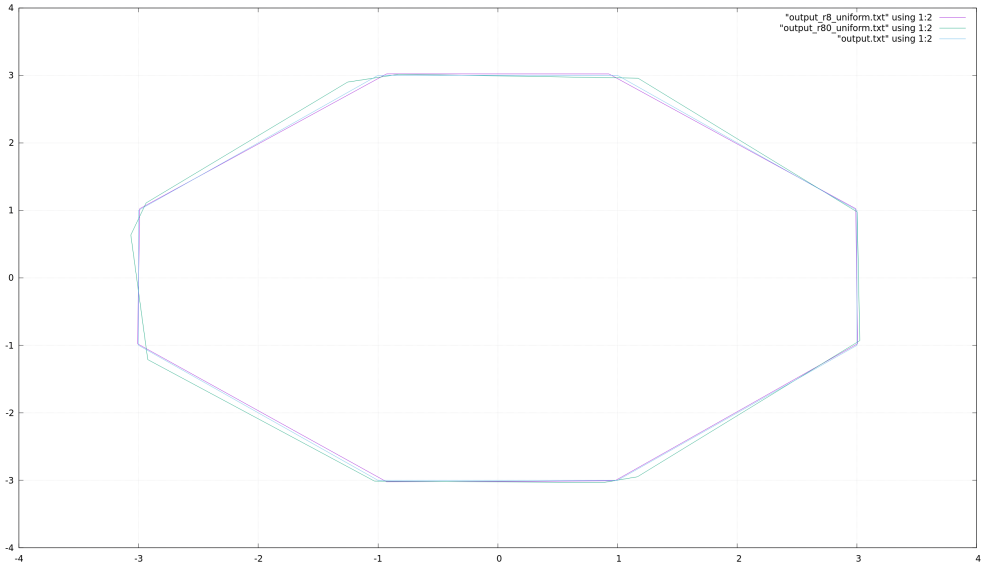


Рис. 12: Сравнение восстановленных методом "ближайших соседей" многоугольников с эталоном

Разница с эталоном для r8:

$$\text{MSE} = 0.00593976327080012$$

$$d_{\max} = 0.07730644761395526$$

Разница с эталоном для r80:

$$\text{MSE} = 0.008201129670586112$$

$$d_{\max} = 0.12613946706363163$$

Разница r8 и r80 (рост ошибок):

$$\text{MSE} = 0.000653811595547378$$

$$d_{\max} = 0.05133493482377407$$

Для метода "ближайших соседей":
Разница с эталоном для r8:

$$\text{MSE} = 0.0022225458187906766$$

$$d_{\max} = 0.07368993797203806$$

Разница с эталоном для r80:

$$\text{MSE} = 9.809287370118973$$

$$d_{\max} = 4.555332917009146$$

Разница r8 и r80 (рост ошибок):

$$\text{MSE} = 9.85429007201206$$

$$d_{\max} = 4.618806375149211$$

Заметим, что метод ближайших соседей может быть улучшен, особенно для симметричных фигур. Пусть функция $r(\alpha)$ является периодической и содержит выраженные экстремумы (максимумы и минимумы), которые естественным образом разбивают её на интервалы.

Разобьём область определения $\alpha \in [0, 2\pi]$ на зоны, границами которых служат ближайшие минимумы функции. В каждой такой зоне локализуются соответствующие максимумы.

Для симметричных фигур (например, восьмиугольника) число таких зон равно числу осей симметрии, то есть 8, и функция $r(\alpha)$ представляется как набор локальных сегментов:

$$r(\alpha) = \bigcup_{i=1}^{N} r_i(\alpha), \quad N = 8.$$

Для каждой зоны применяется метод ближайших соседей, при котором число итераций сглаживания k_i подбирается индивидуально:

$$k_i = \arg\min_k E_i(k) \geq 5 \text{ (нижний порог)}$$

где $E_i(k)$ — ошибка совпадения максимумов в i-й зоне между зашумлённой и эталонной функцией.

Дополнительно вводится условие согласованности: если минимальная ошибка совпадения максимумов равна $e_{\min}$, то она распространяется на все зоны:

$$e_i = e_{\min}, \quad \forall i \in \{1, \ldots, N\}.$$

Таким образом, итоговый критерий выбора оптимального числа итераций определяется как:

$$K^* = \arg\min_{k_1,\ldots,k_N} \sum_{i=1}^{N} E_i(k_i), \quad \text{при условии} \quad e_i = e_{\min}.$$

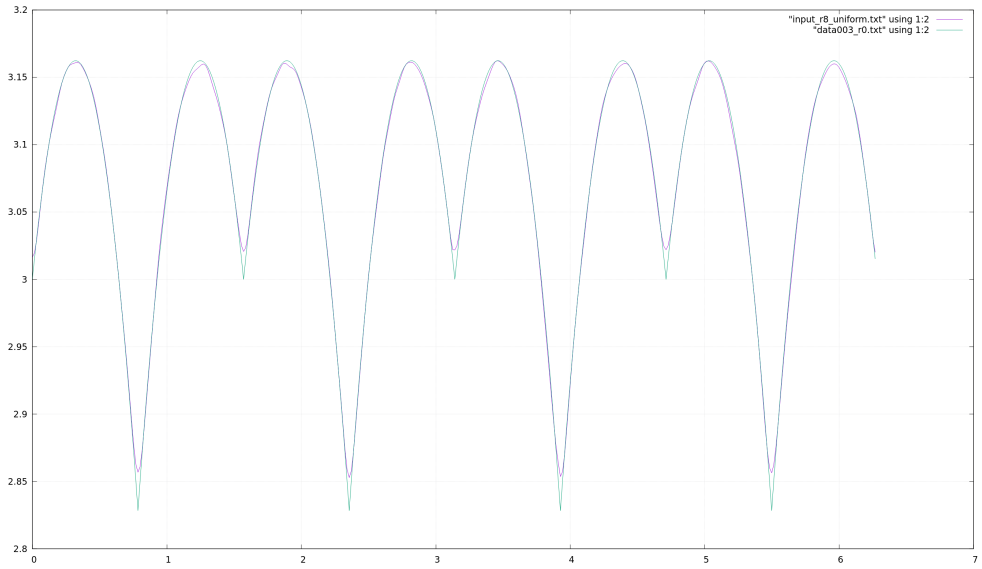


Рис. 13: Модифицированный метод "ближайших соседей" для r8 ($k_i = [5,5,5,5,7,5,5,5]$, $e_{min} \approx 0.001$)

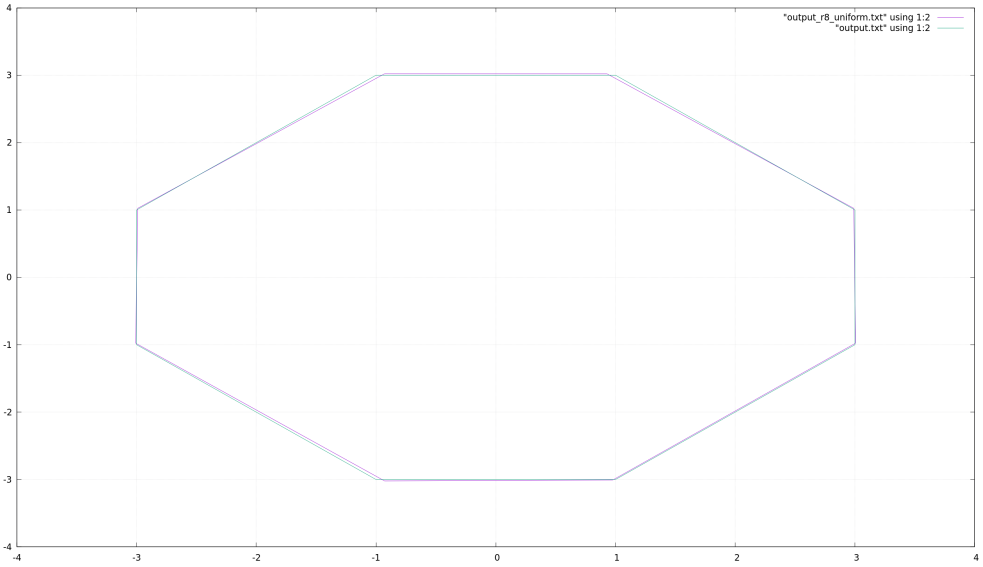


Рис. 14: Сравнение с эталоном

Для модифицированного метода "ближайших соседей":
Разница с эталоном для r8:

$$\text{MSE} = 0.0022224323547445 29$$

$$d_{\max} = 0.07368993797203806$$

Результаты тестирования в 3D ниже (визуализация — *visualization.py*).

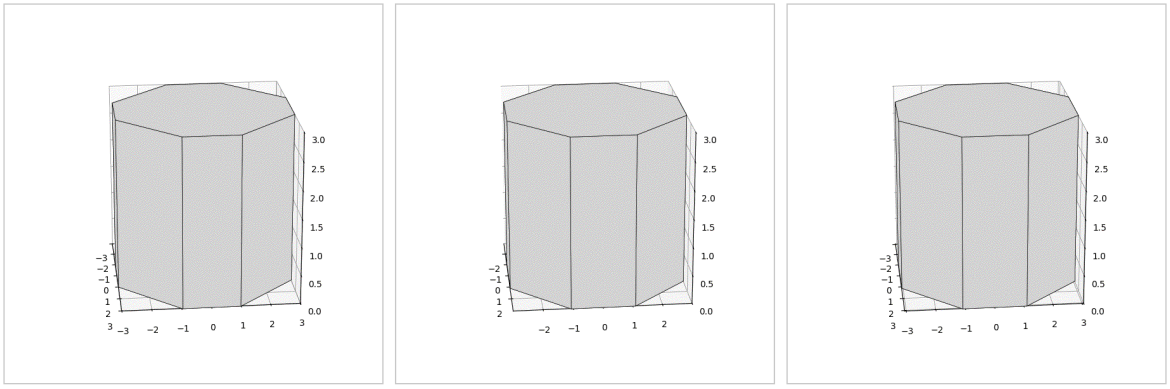


Рис. 15: Призма: эталон (16 вершин, 10 граней) и сглаженные r8- и r80-модели

Для каждой пары точек вычислялось евклидово расстояние:

$$d_i = \sqrt{(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (z_i - \hat{z}_i)^2},$$

где:

- (x_i, y_i, z_i) — точка точной модели,
- $(\hat{x}_i, \hat{y}_i, \hat{z}_i)$ — восстановленная точка.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (z_i - \hat{z}_i)^2\right]}$$

Разница с 3D-эталоном для r8:

$$\text{MSE} = 0.010425178540945794$$
$$\text{RMSE} = 0.10210376359834046$$
$$d_{\max} = 0.10215679062837872$$

Разница с 3D-эталоном для r80:

$$\text{MSE} = 0.01652071924772692$$
$$\text{RMSE} = 0.12853295004677562$$
$$d_{\max} = 0.12996675489126402$$

Разница r8 и r80 (рост ошибок в 3D):

$$\text{MSE} = 0.003355781266968158$$
$$\text{RMSE} = 0.05792910552535882$$
$$d_{\max} = 0.05993940235644994$$

6 Трехмерные тела: генерация сечений, восстановление и подавление шума

В разделе 5 трехмерный эксперимент был ограничен призмой: все слои имели одинаковый контур, поэтому форма сечения не менялась с высотой и проверялась только согласованность слоев по z. Ниже рассматриваются тела с переменным сечением — правильные многогранники, у которых число вершин и форма контура зависят от высоты. Это показывает, что метод восстанавливает тело целиком, а не один контур, и позволяет сравнить методы подавления шума на разных типах сечений.

6.1 Исходные тела

Рассматриваются три правильных многогранника: октаэдр (6 вершин, 8 граней), икосаэдр (12 вершин, 20 граней) и додекаэдр (20 вершин, 12 граней), а также бриллиант — тело с профилем огранки, у которого сечения меняются наиболее резко. Все тела выпуклые: для выпуклого тела опорная функция однозначно задает форму, а вершины сечения — это локальные максимумы $R(\alpha)$.

Вершины задаются стандартными координатами через золотое сечение $\varphi = (1+\sqrt{5})/2$:

$$\begin{aligned}
\text{октаэдр: } & (\pm 1,0,0),\ (0,\pm 1,0),\ (0,0,\pm 1);\\
\text{икосаэдр: } & (0,\pm 1,\pm\varphi),\ (\pm 1,\pm\varphi,0),\ (\pm\varphi,0,\pm 1);\\
\text{додекаэдр: } & (\pm 1,\pm 1,\pm 1),\ (0,\pm 1/\varphi,\pm\varphi),\ (\pm 1/\varphi,\pm\varphi,0),\ (\pm\varphi,0,\pm 1/\varphi).
\end{aligned}$$

Каждое тело нормируется по описанной сфере (радиус 1), равномерно масштабируется до высоты $H = 3$ и сдвигается по z в отрезок $[0,H]$. Тело ставится вершиной вверх. Поворот не меняет геометрию, но избавляет от вырожденных сечений: если сверху оказывается горизонтальное ребро, у полюсов сечение превращается в узкую полоску, соседние вершины сближаются по углу и перестают разрешаться дискретной сеткой. После такой ориентации минимальный угловой зазор между соседними вершинами сечения составляет 90° у октаэдра, 2.3° у икосаэдра, 6.9° у додекаэдра и 36° у бриллианта, то есть во всех случаях больше шага сетки 0.5°.

Четвертое тело — бриллиант короной вверх. Он не является правильным многогранником, но повторяет профиль огранки: площадка (радиус 0.8, высота 3), корона, рундист (радиус 1.7, высоты 2.05 и 1.95), павильон и колет (радиус 0.1, высота 0). Огранка десятигранная: 40 вершин и 32 грани, все горизонтальные сечения — десятиугольники. Тело интересно тем, что ширина сечения сначала растет, затем резко падает, а у колета сечение становится совсем маленьким — там относительный вклад шума максимален.

Таблица 1: Параметры исследуемых тел

Тело	Вершин	Граней	Вершин в сечении
Октаэдр	6	8	4
Икосаэдр	12	20	5–10
Додекаэдр	20	12	3–6
Бриллиант	40	32	10

6.2 Генерация горизонтальных точечных сечений

Слои задаются с шагом $\Delta z = 0.05$:

$$z_k = \left(k + \tfrac{1}{2}\right)\Delta z, \qquad k = 0, \ldots, 59,$$

то есть 60 сечений; полушаг сдвига гарантирует, что плоскость не проходит через вершину тела и сечение не вырождается в точку или ребро.

Для каждого слоя строится сечение тела плоскостью $z = z_k$, его вершины обозначим $P_j^{(k)} = (x_j^{(k)}, y_j^{(k)})$. Направления наблюдения задаются дискретно:

$$\alpha_i = ih, \qquad h = \frac{2\pi}{n}, \qquad n = 720,$$

то есть с шагом $h = 0.5^\circ$. Шаг выбран минимальным из практически приемлемых: он в 4.6 раза меньше наименьшего углового зазора между вершинами сечения (2.3° у икосаэдра), поэтому все вершины разрешаются, а объем данных остается компактным.

Для каждого направления вычисляется опорная функция сечения:

$$R(\alpha_i, z_k) = \max_j \left(x_j^{(k)} \cos\alpha_i + y_j^{(k)} \sin\alpha_i \right).$$

Шум добавляется к каждому значению:

$$\widetilde{R}(\alpha_i, z_k) = R(\alpha_i, z_k) + \varepsilon, \qquad \varepsilon = r_v \xi, \qquad \xi \in [-1, 1],$$

с теми же амплитудами, что и в 2D: $r_v = 0.008$ (r8) и $r_v = 0.08$ (r80). Шум независим для каждой пары (α_i, z_k) и моделирует ошибки измерений, дискретизацию и неточности восстановления контуров.

В результате для каждого тела и каждого уровня шума формируется файл из $60 \times 720 = 43\,200$ строк вида

$$(\alpha_i,\ \widetilde{R}(\alpha_i, z_k),\ z_k),$$

который подается на вход программе *reconstruct3D_layers.py*. Всего девять файлов: $data3D_\{octa, icosa, dodeca\}_\{r0, r8, r80\}.txt$. Генерация выполняется программой *generate3D_bodies.py*.

6.3 Восстановление эталонных тел

Восстановление выполняется по слоям независимо: внутри слоя для каждой точки i считаются разности

$$d_1 = r_i - r_{i-1}, \qquad d_2 = r_{i+1} - r_i,$$

и точка считается вершиной, если $d_1 \geq 0$ и $d_2 < 0$. Координаты вершины:

$$x_i = r_i \cos\alpha_i, \qquad y_i = r_i \sin\alpha_i,$$

а z берется из слоя. Совокупность вершин всех слоев — облако точек, по которому строится выпуклая оболочка; визуализация выполняется программой *visualize3D_bodies.py*.

Программа *reconstruct3D_layers.py* повторяет логику *reconstruct.py*, но обрабатывает каждый слой отдельно: исходный *reconstruct.py* рассчитан на один контур и при подаче многослойного файла замкнул бы массив через границу слоев. Критерий поиска вершин при этом не меняется.

На точных данных (r0) восстанавливается 240 точек для октаэдра, 430 для икосаэдра, 312 для додекаэдра и 600 для бриллианта. Сравнение с аналитическими сечениями дает $\mathrm{MSE} \leq 9.4 \cdot 10^{-7}$: эталон восстанавливается практически точно, остаточная погрешность — только ошибка дискретизации по углу. Эталонные тела показаны на рисунке 16.

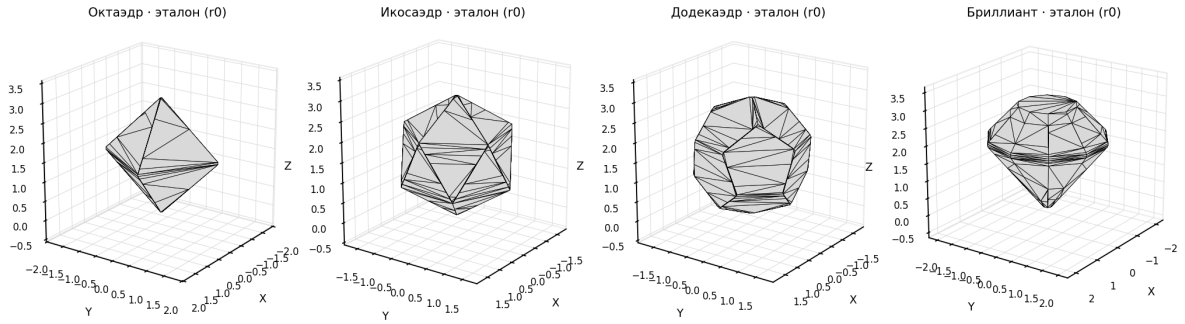


Рис. 16: Эталонные тела, восстановленные по точным сечениям: октаэдр, икосаэдр, додекаэдр, бриллиант. Анимация вращения: body3D_{тело}_r0.gif

6.4 Восстановление зашумленных моделей

Если подавление шума не выполнять, шумовые всплески сами становятся локальными максимумами. Число восстановленных точек вырастает с 240–600 до 12–14 тысяч: 12 493 (октаэдр), 13 204 (икосаэдр), 12 235 (додекаэдр) и 13 830 (бриллиант) при r8; 14 374, 14 388, 14 382 и 14 465 при r80. Оболочка покрывается тысячами мелких граней и «лохматится», а тело слегка увеличивается: шум дает смещения наружу, и максимум по слою выбирает самые выступающие точки. Ошибка при этом растет: MSE равен 0.00378 / 0.00140 / 0.00392 / 0.000293 для r8 и 0.01334 / 0.00912 / 0.01358 / 0.006382 для r80 (октаэдр / икосаэдр / додекаэдр / бриллиант). Зашумленные модели показаны на рисунке 17.

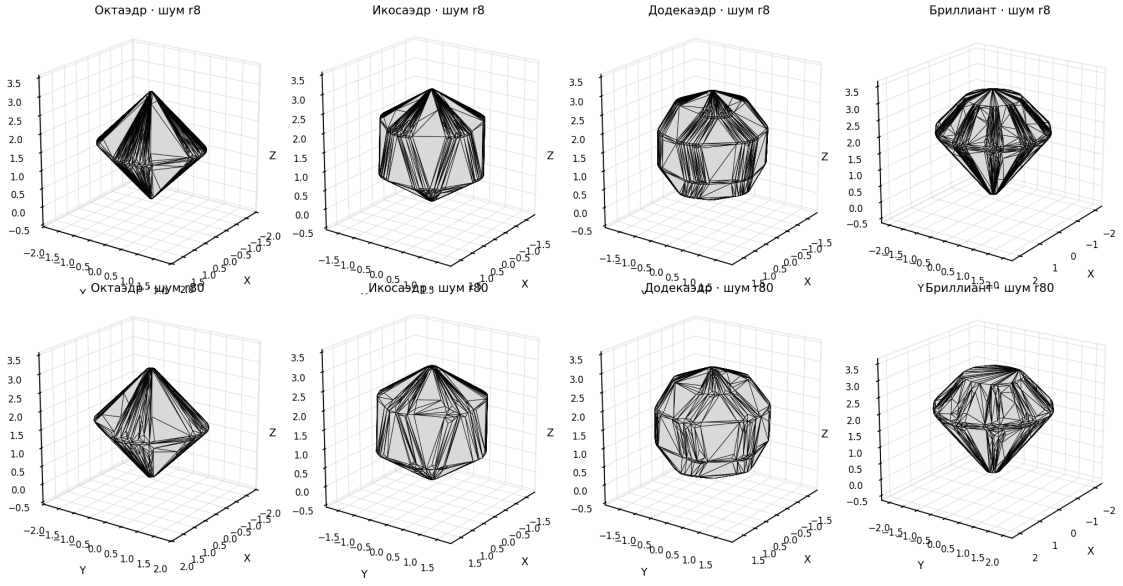


Рис. 17: Зашумленные модели: сверху r8, снизу r80. Анимация вращения: body3D_{тело}_r8.gif и body3D_{тело}_r80.gif

6.5 Подавление шума по слоям

Сглаживание применяется к каждому слою отдельно: функция $r(\alpha)$ определена в плоскости $z = z_k$, и смешивать значения разных высот нельзя.

Ряд Фурье и МНК. Для каждого слоя

$$r(\alpha) \approx a_0 + \sum_{m=1}^{M} \left(a_m \cos m\alpha + b_m \sin m\alpha\right),$$

коэффициенты находятся из системы $Ac \approx r$ с матрицей

$$A = \begin{bmatrix} 1 & \cos\alpha_1 & \sin\alpha_1 & \dots \\ 1 & \cos\alpha_2 & \sin\alpha_2 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}, \qquad c = \begin{bmatrix} a_0 & a_1 & b_1 & \dots \end{bmatrix}^T,$$

из условия $\|Ac - r\|^2 \to \min$, то есть

$$c = (A^T A)^{-1} A^T r.$$

Число гармоник принято $M = 12$. Оно выбрано по максимальному числу вершин сечения: у икосаэдра сечение содержит до 10 вершин, а опорная функция многоугольника имеет изломы в направлениях нормалей к ребрам, поэтому ряд должен разрешать не только вершины, но и углы. Проверка при $M = 8$ на тех же данных дала MSE в 2.4 раза хуже для икосаэдра (0.00180 против 0.00077 при r80) и почти не изменила результат для додекаэдра, где сечение содержит не более 6 вершин (0.00104 против 0.00099). В 2D при восьмивершинном сечении использовалось $M = 8$ — то же правило: число гармоник не меньше числа вершин.

Метод «ближайших соседей». Для каждого слоя

$$\widetilde{r}(\alpha_i) = \frac{2r(\alpha_i) + r(\alpha_{i-1}) + r(\alpha_{i+1})}{4},$$

и это усреднение применяется k раз. Как и в 2D, область разбивается на зоны между соседними минимумами, и для каждой зоны подбирается свое $k_i \in [5, 30]$ по критерию совпадения максимумов с эталоном: $k_i = \arg\min_k E_i(k)$, нижний порог $k = 5$. Подбор k_i использует эталонную функцию, поэтому в этой постановке метод соседей настроен по эталону, тогда как МНК использует только число гармоник.

Результаты сглаживания показаны на рисунках 18 и 19.

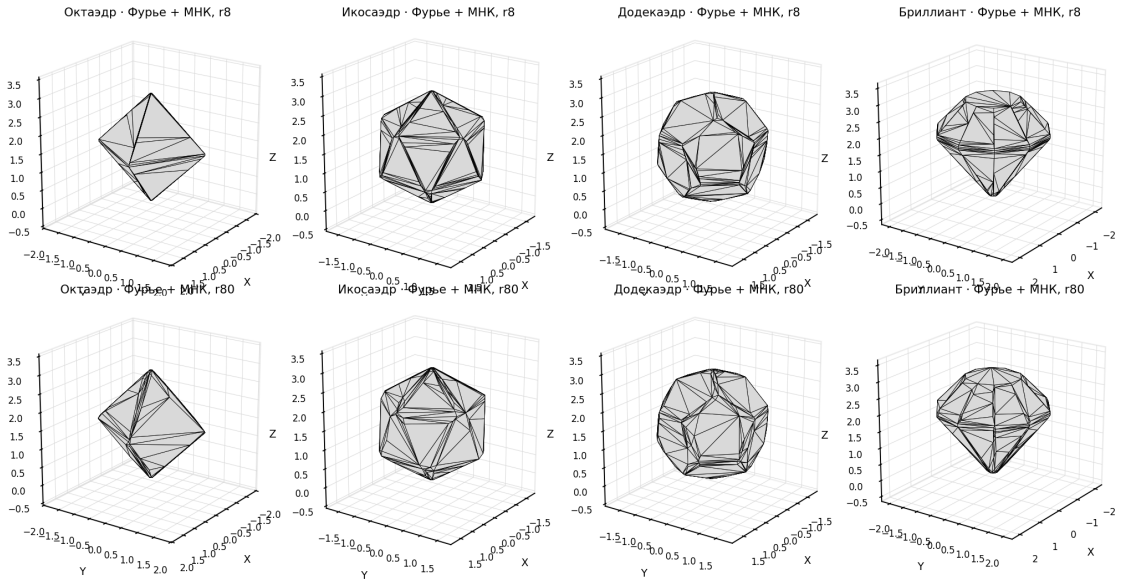


Рис. 18: После подавления шума методом Фурье и МНК: сверху r8, снизу r80. Анимация: body3D_{тело}_fft_r8.gif и body3D_{тело}_fft_r80.gif

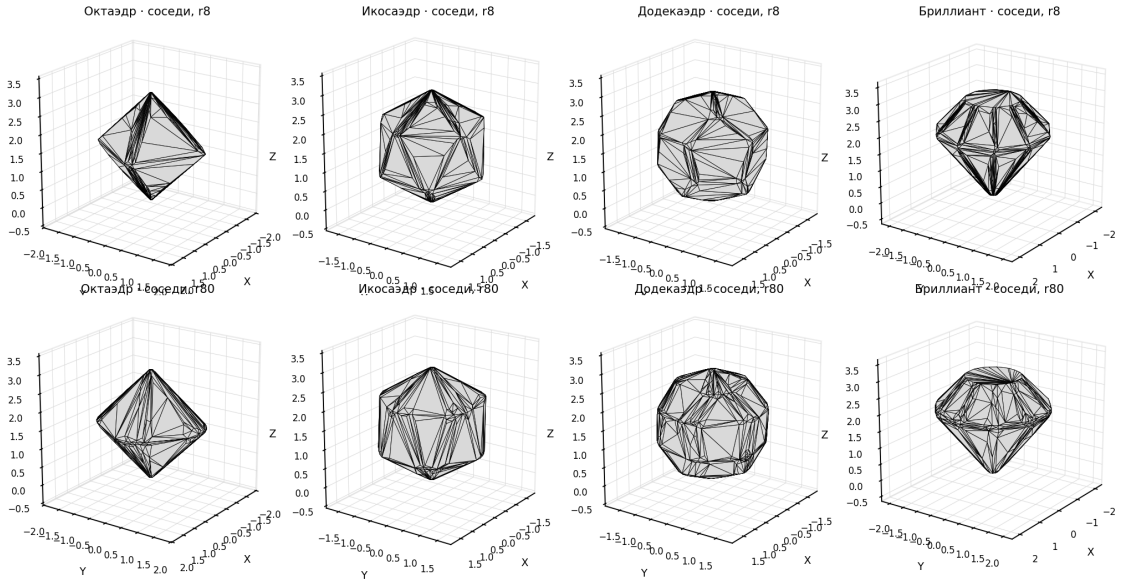


Рис. 19: После подавления шума методом «ближайших соседей»: сверху r8, снизу r80. Анимация: body3D_{тело}_uni_r8.gif и body3D_{тело}_uni_r80.gif

6.6 Сравнение с эталоном

Сравнение выполняется по опорной функции: для каждого слоя и каждого направления сетки считается отклонение

$$\varepsilon(\alpha_i, z_k) = \widehat{R}(\alpha_i, z_k) - R(\alpha_i, z_k),$$

$$\mathrm{MSE} = \frac{1}{N}\sum \varepsilon^2, \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}, \qquad d_{\max} = \max|\varepsilon|.$$

Такой критерий не зависит от числа восстановленных вершин (после шума оно меняется) и напрямую измеряет отклонение формы. Результаты приведены в таблицах 2 и 3.

Таблица 2: MSE относительно аналитического эталона

Тело	r8	r80	Фурье r8	Фурье r80	Соседи r8	Соседи r80
Октаэдр	0.00378	0.01334	0.000014	0.000290	0.000563	0.00418
Икосаэдр	0.00140	0.00912	0.000205	0.000766	0.000271	0.00176
Додекаэдр	0.00392	0.01358	0.000380	0.000990	0.000568	0.00431
Бриллиант	0.000293	0.006382	0.000016	0.000149	0.000060	0.000508

Таблица 3: Максимальное отклонение $d_{\max}$

Тело	r8	r80	Фурье r8	Фурье r80	Соседи r8	Соседи r80
Октаэдр	0.223	0.288	0.013	0.095	0.115	0.238
Икосаэдр	0.135	0.203	0.105	0.165	0.089	0.154
Додекаэдр	0.252	0.313	0.061	0.194	0.146	0.265
Бриллиант	0.049	0.120	0.011	0.148	0.038	0.074

Главный результат: МНК снижает MSE в 7–266 раз относительно зашумленной модели, метод «ближайших соседей» — в 3–13 раз. При этом МНК возвращает и правильное число вершин: 247–600 точек против 240–600 у эталона, тогда как после соседей остается 871–3787 точек, то есть часть ложных максимумов сохраняется. По MSE МНК точнее соседей в 1.3–40 раз при обоих уровнях шума; при r80 разрыв составляет от 2.3 до 14 раз.

Максимальное отклонение ведет себя иначе. На икосаэдре при r8 метод соседей дает $d_{\max} = 0.089$ против 0.105 у МНК, хотя MSE у МНК в 1.3 раза ниже. Причина в том, что МНК сглаживает сильнее и «срезает» острые углы сечения, а $d_{\max}$ чувствителен к отдельным направлениям; по интегральной мере ошибки МНК точнее.

Отдельно отметим роль числа гармоник. При $M = 8$, как в 2D, МНК работал хуже на десятиугольных сечениях икосаэдра: ряда не хватало, чтобы передать углы, и ошибка росла. Увеличение M до 12 снимает это ограничение. Практическое правило: число гармоник должно быть не меньше максимального числа вершин сечения, а лучше — с запасом, поскольку опорная функция многоугольника имеет изломы.

Итог раздела: на телах с переменным сечением оба метода восстанавливают форму, но МНК точнее по интегральной ошибке, устойчивее к росту шума и возвращает правильное число вершин; метод «ближайших соседей» проще, не требует выбора числа гармоник и в отдельных случаях дает меньшее максимальное отклонение, однако оставляет ложные вершины. Все анимации вращения для каждого тела и каждого варианта сохранены в каталоге out3D (файлы *body3D_{тело}_{вариант}.gif*).

6.7 Минимальный многогранник: автоматический режим

По облаку точек можно строить не только выпуклую оболочку, но и минимальный многогранник — модель с наименьшим числом граней, сохраняющую форму. В программе *visualization.py* для этого предусмотрен автоматический режим: по оболочке выполняется квадратичная декцимация, доля оставшихся треугольников подбирается двоичным поиском так, чтобы площадь поверхности уменьшилась не более чем на 0.1%, а затем соседние треугольники с близкими нормалями (угол до 1°) сливаются в многоугольные грани. Число гармоник выбирается по источнику: 12 для демонстрационных тел и 32 для реальных моделей. Результат задается двумя числами — числом вершин и числом граней.

Поясним термин. Декцимация (от латинского *decimatio* — «убавление на десятую часть»; в 3D-графике — прореживание) это упрощение сетки: часть вершин и треугольников удаляется, а форма сохраняется. В русскоязычной литературе тот же прием чаще называют «упрощением сетки» или «понижением детализации». В квадратичной декцимации каждой вершине

сопоставляется цена удаления — сумма квадратов расстояний до плоскостей соседних граней; алгоритм последовательно стягивает ребро с наименьшей ценой и пересчитывает цену соседей, пока не останется заданное число треугольников. Поэтому «минимальный» многогранник — это не произвольное огрубление, а наименее искажающее форму при заданном числе граней.

В приведенных ниже результатах после каждой декцимации оболочка перестраивается заново, и площадь для критерия 99.9% считается уже по ней; это убирает уступы на плоских гранях (подробнее — в разделе 8).

Автоматический режим применен ко всем 28 случаям: четырем телам и семи вариантам восстановления. Результаты приведены в таблице 4, в ячейке указано число вершин и граней.

Таблица 4: Минимальный многогранник: число вершин / граней (автоматический режим)

Тело	r0	r8	r80	Фурье r8	Фурье r80	Соседи r8	Соседи r80
Октаэдр	6/8	7/10	135/228	6/8	6/8	6/8	6/8
Икосаэдр	12/20	14/24	14/24	12/20	12/20	12/20	12/20
Додекаэдр	20/12	244/438	18/32	19/26	19/29	18/29	18/28
Бриллиант	17/25	258/417	228/370	14/24	13/22	14/22	14/23

На эталонных данных режим точно восстанавливает октаэдр (6 вершин, 8 граней) и икосаэдр (12 вершин, 20 граней), причем после любого из двух методов сглаживания числа совпадают с исходными. Для додекаэдра эталон также восстанавливается точно (20 вершин, 12 граней), но после сглаживания получается 18–19 вершин и 26–29 граней: пятиугольные грани аппроксимируются несколькими почти компланарными треугольниками, и порог слияния 1° их не объединяет. У бриллианта теряются мелкие элементы — рундист и колет: их площадь мала, и критерий по площади их срезает (17 вершин и 25 граней вместо 40 и 32).

На зашумленных данных картина зависит и от уровня шума, и от размера исходных граней. У октаэдра грани крупные, поэтому при r8 шум не создает устойчивых фасетов и минимальный многогранник сводится к 7 вершинам и 10 граням. У додекаэдра грани мелкие, и уже при r8 сохраняются 244 вершины и 438 граней; при r80 у октаэдра — 135 вершин и 228 граней. Отметим, что у додекаэдра при r80 число граней падает до 32: автоматический порог по площади выбирает более грубую модель (2.8% треугольников против 27% при r8), то есть критерий по площади не монотонен по уровню шума.

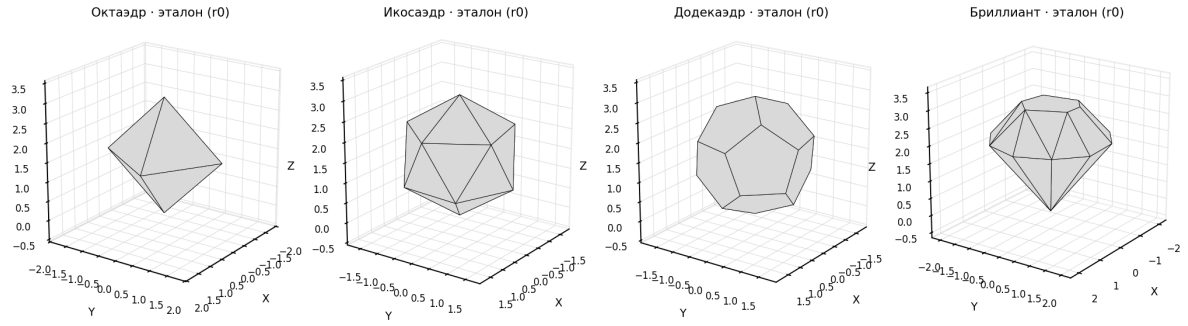


Рис. 20: Минимальные многогранники по точным сечениям: октаэдр (6/8), икосаэдр (12/20), додекаэдр (20/12), бриллиант (17/25)

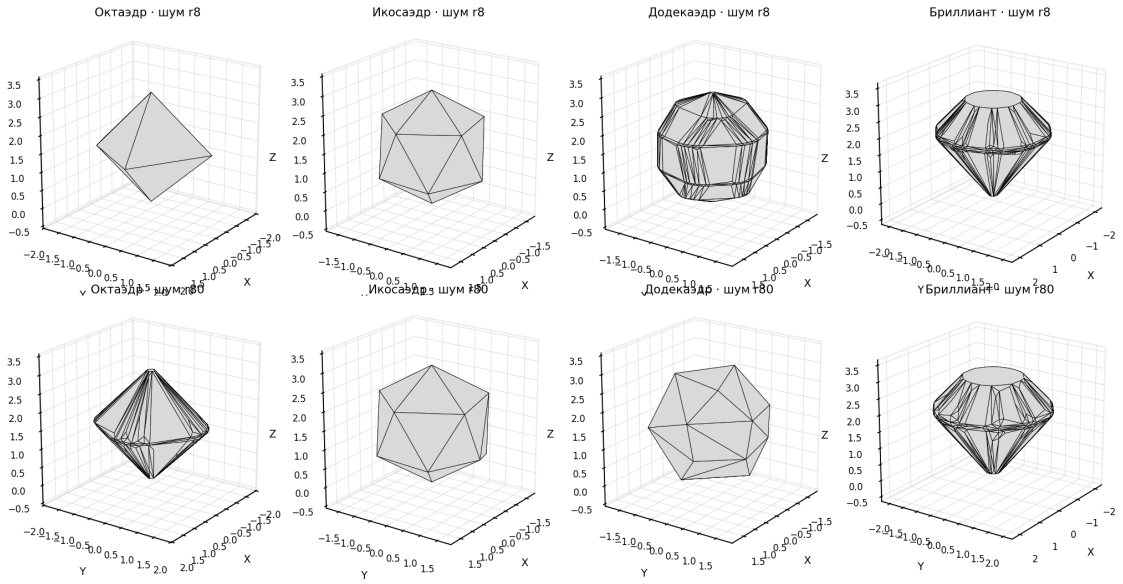


Рис. 21: Минимальные многогранники для зашумленных моделей: сверху r8, снизу r80

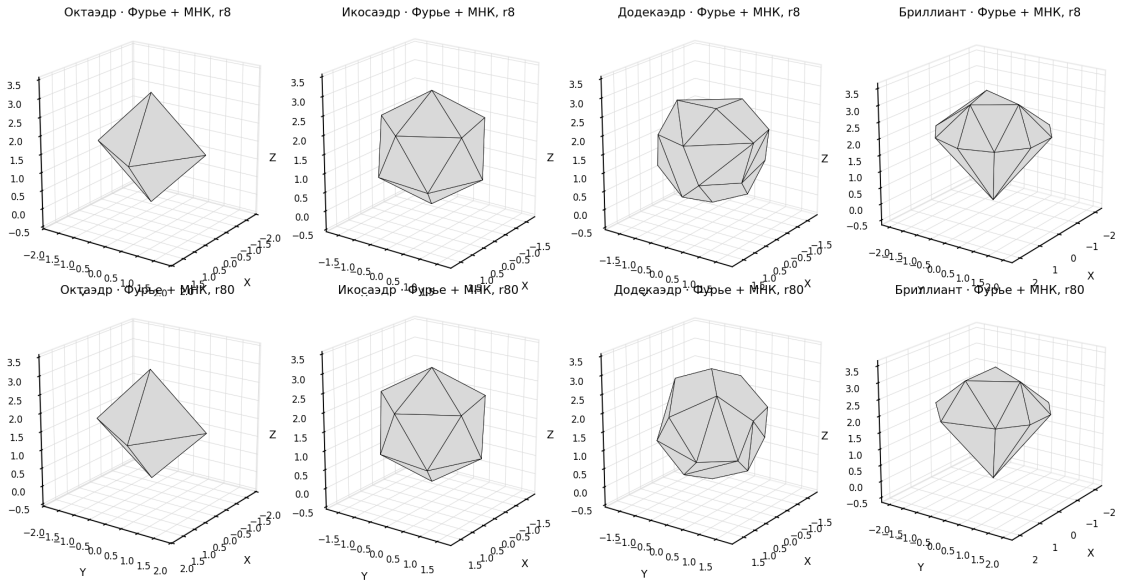


Рис. 22: Минимальные многогранники после подавления шума методом Фурье и МНК: сверху r8, снизу r80

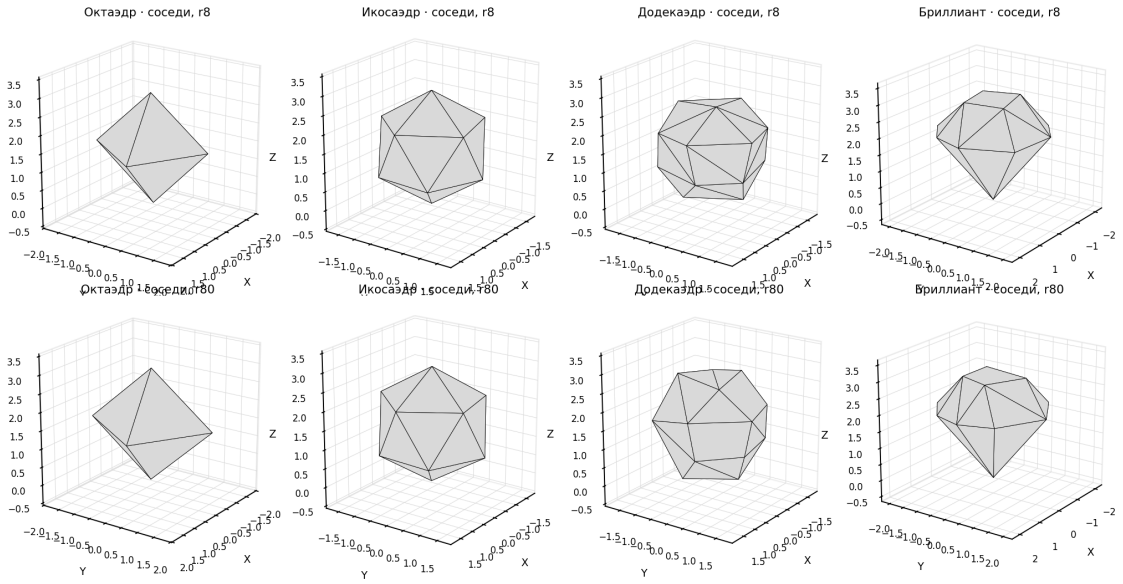


Рис. 23: Минимальные многогранники после подавления шума методом «ближайших соседей»: сверху r8, снизу r80

Вывод: автоматический режим дает компактное представление формы и точно восстанавливает тела с треугольными и четырехугольными гранями, но чувствителен к допуску по площади и порогу компланарности; для мелких и пятиугольных граней нужен меньший допуск. Анимации минимальных многогранников сохранены в каталоге out3D (файлы *min3d_{тело}_{вариант}.gif*).

Все модели — точные тела, оболочки и минимальные многогранники — выгружены в формат OBJ в каталог $out3D/obj$: 60 файлов и манифест $manifest.csv$ с числом вершин и граней. OBJ открывается в Blender, MeshLab и большинстве 3D-редакторов; для просмотра без сторонних программ служит скрипт $view3D.py$, который строит модель в окне с вращением мышью.

7 Реальные данные

До сих пор все тела были синтетическими. Ниже тот же конвейер применяется к трем моделям, полученным сканером: $Reflect_00001$, $Reflect_10001$ и $Reflect_20001$.

7.1 Данные и подготовка

Модели заданы выпуклыми многогранниками: 552, 436 и 490 вершин и 278, 220 и 247 граней соответственно. Габариты до нормировки: $6.4 \times 6.4 \times 4.0$, $4.5 \times 8.3 \times 2.5$ и $5.8 \times 3.9 \times 2.2$. Все вершины лежат на выпуклой оболочке, то есть тела выпуклые. В файлах площадка расположена внизу, а колет сверху, поэтому модели переворачиваются: корона оказывается наверху, высота нормируется к $H = 3$, z отсчитывается от нуля.

Сечения строятся так же: 60 слоев с шагом 0.05 и 720 направлений с шагом 0.5°. Реальные сечения устроены сложнее синтетических: в них от 8 до 217 вершин (в среднем около 30), а минимальный угловой зазор между соседними вершинами достигает 0.02° — это микрограни сканера, которые дискретная сетка не разрешает. Поэтому сечения считаются по ребрам сетки, а не по граням.

7.2 Два режима восстановления

Исходный прием — считать вершины локальными максимумами $R(\alpha)$ — опирается на то, что каждая вершина многоугольника является радиальным максимумом. Для правильных многогранников это так, для реальных огранок — нет: у гладких овальных сечений радиальных максимумов всего 3–4 на слой. Проверка это подтверждает: на точных сечениях режим максимумов дает 1294 точки для первого образца (MSE $4.1 \cdot 10^{-5}$), но лишь 244 и 293 точки для второго и третьего, а ошибка возрастает до 0.12 и 0.077.

Поэтому для реальных данных используется обобщение: сечение восстанавливается как пересечение полуплоскостей

$$\widehat{K} = \{x : \ x \cdot e(\alpha_i) \le R(\alpha_i) \ \ \forall i\}, \qquad e(\alpha) = (\cos\alpha, \ \sin\alpha)^T,$$

то есть как тело, для которого измеренные значения $R(\alpha_i)$ являются опорными. На точных данных это обращение точное: MSE составляет 10^{-13}, а максимальное отклонение — 10^{-6}.

Таблица 5: Точные сечения: режим максимумов и обобщенное восстановление

Образец	Максимумы (точек / MSE)	Полуплоскости (точек / MSE)	min (вершин / граней)
1 ($Reflect_00001$)	1294 / $4.1 \cdot 10^{-5}$	43 200 / $4.8 \cdot 10^{-14}$	95 / 147
2 ($Reflect_10001$)	244 / 0.1197	41 883 / $2.3 \cdot 10^{-13}$	95 / 149
3 ($Reflect_20001$)	293 / 0.0774	42 854 / $2.0 \cdot 10^{-13}$	101 / 151

Обобщенное восстановление требует 720 точек на слой, поэтому минимальный многогранник получается крупнее, чем у синтетических тел: 95–101 вершина и 147–151 грань против 436–552 вершин и 220–278 граней исходной модели. Исходные модели и восстановленные по точным сечениям тела показаны на рисунках 24 и 25.

Автоматический режим (раздел 6.7) применен ко всем вариантам реальных данных: на рисунках этого раздела показаны именно минимальные многогранники, а не триангулированные оболочки. Для точных сечений они содержат 95–101 вершину и 147–151 грань; после сглаживания — от 392 до 584 вершин и от 584 до 676 граней, потому что критерий по площади сохраняет мелкие грани сглаженной оболочки.



Рис. 24: Реальные модели сканера: образец 1, образец 2, образец 3 (короной вверх, высота нормирована)



Рис. 25: Восстановление по точным сечениям (пересечение полуплоскостей); показан минимальный многогранник: ошибка MSE $\leq 2.3 \cdot 10^{-13}$

7.3 Зашумленные данные

На реальных данных шум проявляется иначе, чем на синтетических. Зашумленные значения $R(\alpha_i)$ несовместны: не существует выпуклого тела, для которого все они были бы опорными. Поэтому пересечение полуплоскостей вырождается: при r8 остается 259–407 точек (MSE 0.32–0.55), при r80 — от 4 до 25 точек (MSE 2.17–5.38), а у третьего образца восстановление вырождается полностью. Иными словами, без сглаживания обобщенное восстановление невозможно, и на реальных данных сглаживание обязательно, а не желательно.

7.4 Сглаживание

Сглаживание выполняется по слоям. Число гармоник увеличено до $M = 32$: реальные сечения содержат десятки микро-граней (типичное значение около 30), и ряд должен их разрешать; экстремальные значения до 217 точек считаются шумом сканера.

После Фурье и МНК ошибка падает до $8.4 \cdot 10^{-6}$, $3.2 \cdot 10^{-5}$ и $2.1 \cdot 10^{-5}$ при r8 и до $2.9 \cdot 10^{-4}$, $5.5 \cdot 10^{-4}$ и $5.1 \cdot 10^{-4}$ при r80. Метод «ближайших соседей» при r8 сравним ($5.9 \cdot 10^{-5}$, $1.0 \cdot 10^{-4}$, $5.4 \cdot 10^{-5}$), но при r80 проигрывает на порядок: $3.1 \cdot 10^{-2}$, $2.7 \cdot 10^{-2}$ и $1.5 \cdot 10^{-2}$. Результаты сведены в таблице 6 и показаны на рисунках 26 и 27.

Таблица 6: MSE на реальных данных (режим полуплоскостей)

Образец	r8	r80	Фурье r8	Фурье r80	Соседи r8	Соседи r80
1	0.324	2.17	$8.4 \cdot 10^{-6}$	$2.9 \cdot 10^{-4}$	$5.9 \cdot 10^{-5}$	$3.1 \cdot 10^{-2}$
2	0.550	5.38	$3.2 \cdot 10^{-5}$	$5.5 \cdot 10^{-4}$	$1.0 \cdot 10^{-4}$	$2.7 \cdot 10^{-2}$
3	0.423	—	$2.1 \cdot 10^{-5}$	$5.1 \cdot 10^{-4}$	$5.4 \cdot 10^{-5}$	$1.5 \cdot 10^{-2}$

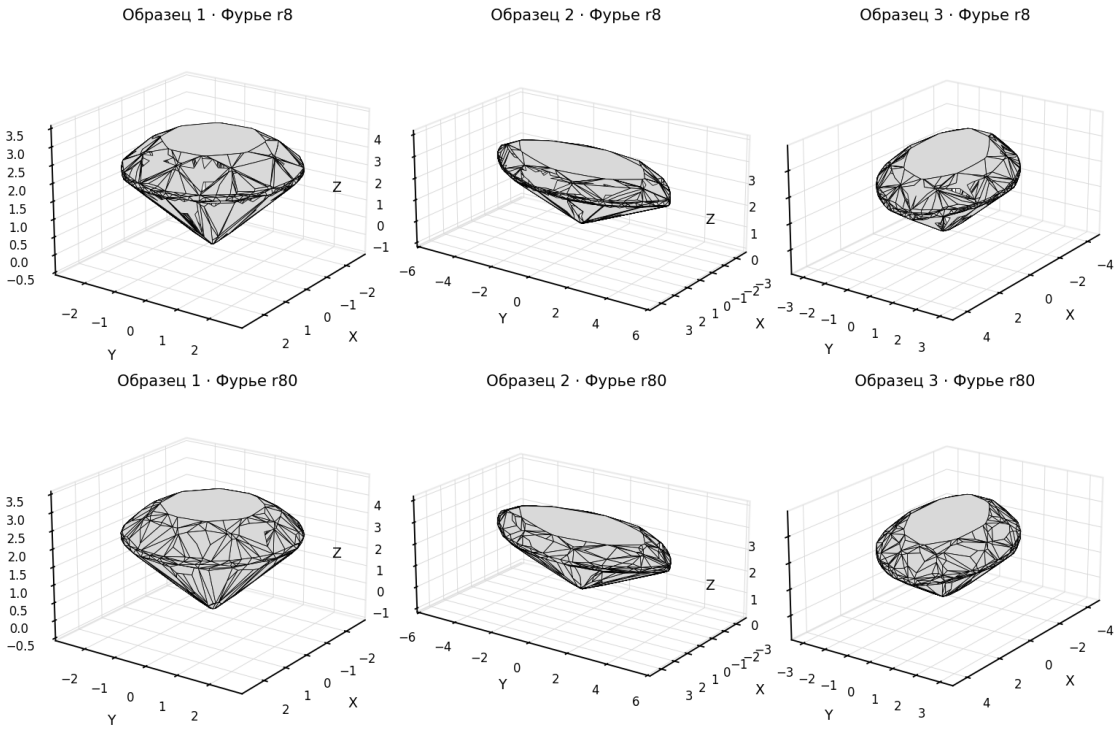


Рис. 26: Реальные модели после подавления шума методом Фурье и МНК (минимальные многогранники): сверху r8, снизу r80

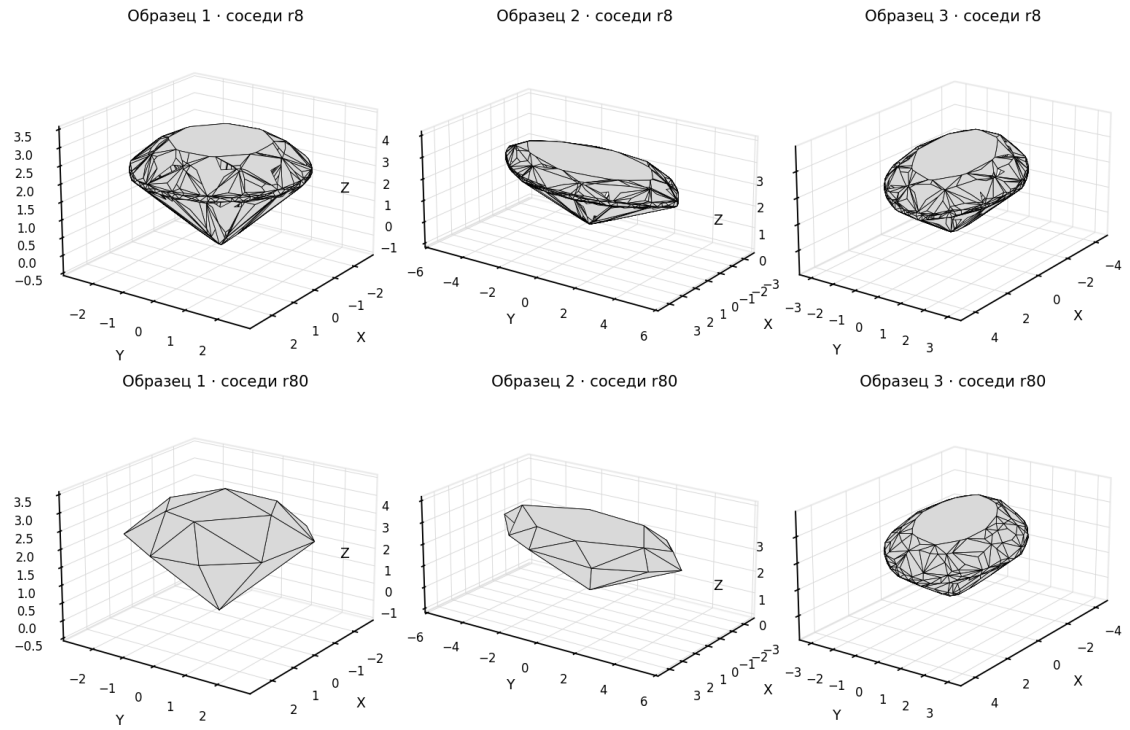


Рис. 27: Реальные модели после подавления шума методом «ближайших соседей» (минимальные многогранники): сверху r8, снизу r80

7.5 Вывод по реальным данным

Реальные данные потребовали двух изменений, которых не было видно на правильных многогранниках: восстановления через пересечение полуплоскостей и обязательного сглаживания. После этого конвейер работает: точные сечения восстанавливаются с ошибкой порядка 10^{-13}, а зашумленные после Фурье и МНК — с ошибкой 10^{-5}–10^{-4}, то есть форма восстанавливается. Метод «ближайших соседей» при умеренном шуме сравним с МНК, но при сильном проигрывает на порядок. Результаты выгружены в OBJ (каталог $out3D_real/obj$): точные модели и минимальные многогранники для всех вариантов.

8 Программа «Огранка»

Для проверки метода на произвольных данных написана программа «Огранка», объединяющая весь конвейер: он принимает модель, строит сечения, добавляет шум, сглаживает, восстанавливает форму, строит минимальный многогранник и показывает результат в интерактивном окне.

Источники данных — четыре демонстрационных тела, файлы OBJ и текстовые файлы координат (x, y, z). В интерфейсе задаются несколько уровней шума одновременно (r8, r80 и произвольные), оба метода сглаживания (для «соседей» — адаптивный подбор $k_i \in [5, 30]$ по зонам), режим восстановления (максимумы или полуплоскости), число слоев и направлений, автоматический минимальный многогранник с ручными параметрами — допуском по площади и углом слияния граней.

Результат показывается в двух–шести окнах одновременно; в каждом выбирается свой вариант, выделенная грань или вершина подсвечивается во всех окнах, а камера может вращаться синхронно. Доступны режимы раскраски (однотонный, по граням, по площади), каркас, рёбра и вершины. Ниже выводятся таблица метрик (MSE, RMSE, $d_{\max}$, число вершин и граней), оценка качества (хорошо, средне, плохо) и пакетная таблица по всем загруженным моделям. Минимальный многогранник строится квадратичной декцимацией Open3D и слиянием компланарных граней, как в разделе 6.7; допуск площади и угол слияния задаются вручную. Модели выгружаются в OBJ, снимок окна — в PNG, таблица — в CSV.

При переносе автоматического режима в «Огранку» обнаружился артефакт, незаметный на грубых моделях. Квадратичная декцимация не сохраняет плоскости исходных фасетов: она не только удаляет вершины, но и сдвигает оставшиеся. Из-за этого плоская грань, например верхняя площадка бриллианта, распадается на несколько почти компланарных треугольников с уступом в сотые доли единицы. На экране это выглядит как скол, а площадь упрощенной модели, посчитанная до перестроения оболочки, не совпадает с площадью ее выпуклой оболочки.

Поэтому после каждой декцимации оболочка строится заново по оставшимся вершинам, а площадь для критерия 99.9% считается уже по ней. Оболочка по определению дает плоские выпуклые фасеты, после чего слияние при 1° собирает их в один многоугольник. Дополнительно вершины возвращаются на плоскости исходных фасетов, если их проекция попадает в контур грани с небольшим запасом: это убирает уступы, которые оболочка иначе сохраняет, поскольку они остаются выпуклыми. На бриллианте верхняя площадка после такой обработки становится одной четырехугольной гранью вместо двух треугольников на разных высотах. На демонстрационных телах с треугольными и пятиугольными гранями результат не меняется: октаэдр — 6 вершин и 8 граней, икосаэдр — 12 и 20, додекаэдр — 20 и 12.

Программа работает локально: сервер на стандартной библиотеке Python и два пакета — numpy и scipy; вьюпорт построен на three.js, который поставляется вместе с программой, поэтому интернет не нужен. Запуск — python3 app.py; для macOS и Windows есть готовые ярлыки. Реальная модель (552 вершины, 60 слоев, 720 направлений, два уровня шума и оба метода сглаживания) обрабатывается примерно за 4 секунды.

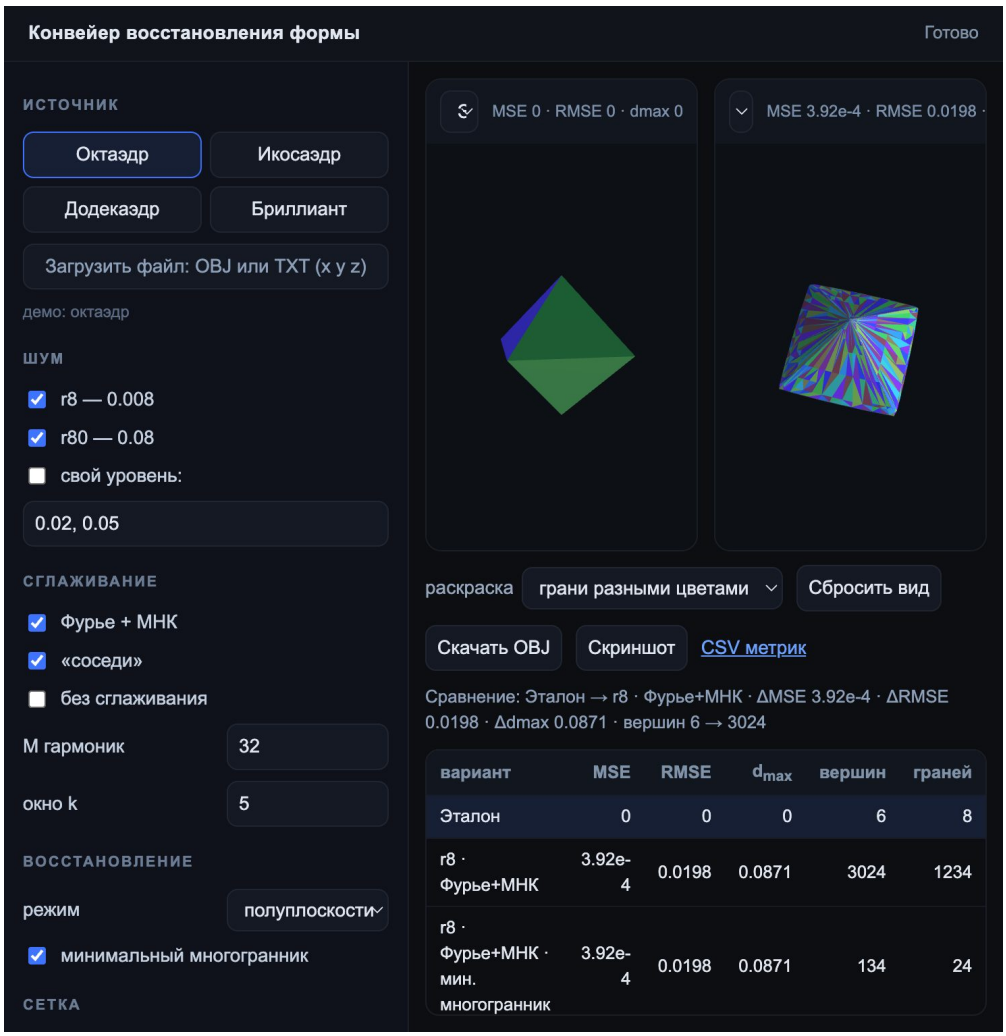


Рис. 28: Окно программы: слева эталон, справа восстановленная модель; раскраска граней и метрики

9 Вывод

Как видно, аппроксимация методом Фурье с МНК эффективно подавляет большую дисперсию, но для умеренной дисперсии лучше подойдет метод "ближайших соседей". В трехмерном эксперименте (раздел 6) при достаточном числе гармоник преимущество МНК сохраняется и при умеренном шуме: он точнее по MSE и возвращает правильное число вершин, тогда как метод "ближайших соседей"остается полезен как более простой и в отдельных случаях дает меньшее максимальное отклонение.

10 Заключение

В работе построен и проверен конвейер восстановления формы выпуклого тела по горизонтальным точечным сечениям: по дискретной опорной функции слоя находятся вершины сечения, из вершин всех слоев собирается облако точек, по которому строится выпуклая оболочка. В 2D метод проверен на восьмиугольнике, в 3D — на четырех телах с переменным сечением: октаэдре, икосаэдре, додекаэдре и бриллианте короной вверх. Для каждого тела сгенерировано 60 сечений с шагом 0.05 по высоте и 720 направлений с шагом 0.5°; шум добавлялся с амплитудами $r_v = 0.008$ (r8) и $r_v = 0.08$ (r80).

На точных данных эталон восстанавливается практически точно: средняя квадратичная ошибка не превышает $9.4 \cdot 10^{-7}$, а отличие от аналитических сечений — только ошибка дискретизации по углу. Без подавления шума число восстановленных вершин вырастает с 240–600 до 12–14 тысяч: каждый шумовой всплеск становится локальным максимумом, оболочка покрывается мелкими гранями, а MSE достигает 0.0136.

Оба метода сглаживания возвращают форму. Аппроксимация рядом Фурье с МНК снижает MSE в 7–266 раз и дает 247–600 вершин — практически столько же, сколько у эталона. Метод «ближайших соседей» снижает MSE в 3–13 раз, но оставляет 871–3787 вершин: часть ложных максимумов сохраняется. По интегральной ошибке МНК точнее в 1.3–40 раз; лишь на икосаэдре при r8 метод соседей дает меньшее максимальное отклонение (0.089 против 0.105), потому что МНК сильнее сглаживает и «срезает» острые углы.

Отдельный методический результат — выбор числа гармоник. Ряд должен разрешать не только вершины, но и углы сечения: при $M = 8$ на десятиугольных сечениях икосаэдра ошибка при r80 составила 0.00180, а при $M = 12$ — 0.00077. Для додекаэдра, где сечение содержит не более шести вершин, увеличение M почти не меняет результат (0.00104 и 0.00099). Правило «M не меньше максимального числа вершин сечения» объясняет и разницу между 2D- и 3D-выводами: в 2D при восьмивершинном сечении $M = 8$ было достаточно, и при умеренном шуме выигрывали соседи; в 3D при достаточном M преимущество МНК сохраняется и при умеренном шуме.

Ограничения метода: он работает для выпуклых тел, чувствителен к выбору числа гармоник и к сильному шуму на малых сечениях (у колета бриллианта относительный вклад шума максимален), а при сильном шуме оставляет ложные вершины. Порог погрешности зависит от геометрической сложности и детализации фигуры.

Наконец, наблюдение, которое стоит за всеми числами: человек и компьютер видят одну и ту же задачу по-разному. Даже если задача поставлена корректно и кажется человеку тривиальной, компьютер может подменять концепции и выполнить то, что по условию не требовалось, либо иначе исказить результат из-за накапливаемых вычислительных погрешностей и отсутствия человеческого опыта. Компьютер может решать одну и ту же задачу несколькими способами, которые человеку кажутся логически абсурдными, — отсюда ошибки и неопределенности: «не увидеть» лишнюю грань, добавить ее или построить многогранник по облаку точек по-разному. Чем сложнее геометрическая форма, тем сложнее, хитрее и продуманнее требуются алгоритмы. Возможно, будущее — за нейросетевыми алгоритмами.

Список литературы

[1] Kak A., Slaney M. Principles of Computerized Tomographic Imaging. http://www.slaney.org/pct/pct-toc.html

[2] Szeliski R. Computer Vision: Algorithms and Applications. Разделы, посвящённые восстановлению формы и силуэтам. https://szeliski.org/Book/

[3] Laurentini A. The Visual Hull Concept for Silhouette-Based Image Understanding // IEEE Transactions on Pattern Analysis and Machine Intelligence. https://ieeexplore.ieee.org/document/273735

[4] CGAL Computational Geometry Library. Материалы по выпуклым оболочкам и вычислительной геометрии. https://www.cgal.org/

[5] Григорьев А. Выпуклая оболочка: алгоритмы построения и анализ. https://habr.com/ru/post/144921/

[6] Сидоренко А.А., Меженин А.В. Обработка облаков точек для повышения качества 3D-моделей. https://kmu.itmo.ru/digests/article/14050

[7] Чурилин О.И., Белов Ю.С. Методы устранения артефактов и шумов при цифровой 3D-реконструкции объектов из изображений. https://library.bmstu.ru/Catalog/Details/596871

[8] Артемьев В.М., Наумов А.О., Кохан Л.Л. Адаптивная фильтрация комплексированных измерений методом наименьших квадратов. http://depositary.belisa.org.by/EDNI/Periodicals/Articles/Details.aspx?Key_Journal=32&Id=965

[9] Голиков Р.Ю., Коростелкина И.А., Ляпина И.Р., Мекшенева Ж.В. Восстановление формы сильно зашумленного сигнала на основе интегро-дифференциального преобразования и аппроксимации интегральной кривой. http://www.appliedinformatics.ru/r/free/article/index.php?article_id_4=3189

[10] Удаление шума и деформация 3D-облаков точек в Python с помощью графовых лапласианов. https://habr.com/ru/articles/1010452/